

Retrieval Augmented Generation RAG

RAG Chunking

dans les Systèmes RAG

Programme 4IASDR

5 méthodes à explorer

01 Fixed-Size Chunking

02 Fixed-Size with Overlap

03 Recursive Character Chunking

04 Semantic Chunking

05 Agentic / Proposition Chunking

Objectifs pédagogiques

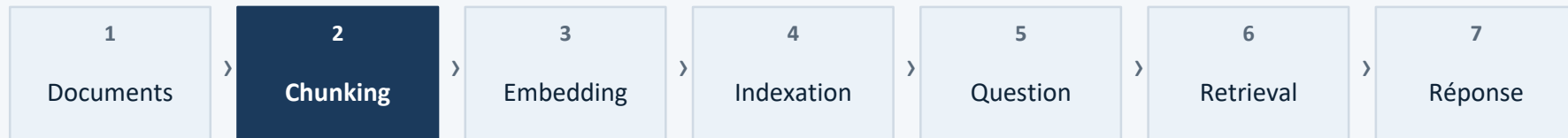
À l'issue de ce cours, l'étudiant sera capable de...

1	Définir le chunking	Comprendre son rôle dans la chaîne RAG et pourquoi il est indispensable.
2	Connaître les propriétés d'un bon chunk	Cohérence sémantique, taille adaptée, continuité du sens.
3	Maîtriser 5 méthodes de chunking	Fixed-Size, Overlap, Recursive, Semantic, Agentic — avec implémentation Python.
4	Choisir la méthode selon le contexte	Sélectionner la stratégie optimale selon le type de document et la précision requise.
5	Intégrer le chunking dans un pipeline RAG	Articuler chunking, embedding, base vectorielle et retrieval de façon cohérente.

I. Contexte — Qu'est-ce que le RAG ?

Retrieval-Augmented Generation : enrichir un LLM avec des sources externes

Le RAG enrichit un LLM avec une base de connaissances externe interrogée dynamiquement — sans réentraîner le modèle. Il résout le problème des hallucinations et de l'information périmée.



↑ *Étape centrale — sa qualité
détermine toute la chaîne*

LLM seul — Limites

- Connaissance figée à la date d'entraînement
- Hallucinations fréquentes sur les faits spécifiques
- Impossible à mettre à jour sans réentraîner

LLM + RAG — Apports

- Sources externes fraîches et vérifiables
- Réponses ancrées dans des documents réels
- Mise à jour simple : modifier la base vectorielle

II. Qu'est-ce que le Chunking ?

Définition et raisons fondamentales

Le chunking est l'opération qui consiste à découper un document long en petits segments — paragraphes, phrases ou blocs de taille fixe — afin de les transformer en embeddings vectoriels et de les indexer efficacement dans une base de données.

Pourquoi est-ce nécessaire ?

Un document entier est trop long pour un seul embedding cohérent

Un long texte mélange plusieurs idées — le vecteur résultant est peu représentatif

Dans un RAG, on veut retrouver la partie précise qui répond à la question

Segmenter permet d'obtenir des embeddings ciblés et précis

Un chunk peut être...

01 Paragraphe

02 Quelques phrases consécutives

03 Bloc de N tokens (taille fixe)

04 Section logique structurée

III. Propriétés d'un bon Chunk

Trois critères à respecter pour un chunking de qualité

Un bon chunk respecte trois propriétés essentielles : cohérence sémantique (un seul sujet), taille raisonnable (ni trop petit ni trop grand), et continuité du sens (aucune phrase brisée).

a) Cohérence sémantique

✓ Correct

Chunk A : « Le RAG utilise un retriever pour sélectionner les passages pertinents. »
Chunk B : « Le LLM génère la réponse en s'appuyant sur les chunks récupérés. »

✗ Incorrect

Chunk C : « Le RAG récupère des passages... les transformers sont des architectures attention... le fine-tuning modifie les poids... »
→ 3 sujets mélangés, embedding incohérent

b) Taille raisonnable

✓ Correct

Chunk idéal (~100 mots) : « L'embedding est une représentation vectorielle dense. Il capture le sens sémantique et permet de mesurer la similarité via la distance cosin. »

✗ Incorrect

Trop petit : « L'embedding est un vecteur. »
→ Trop pauvre pour l'embedding
Trop grand : définition + fonctionnement + limites → Vecteur dilué, peu discriminant

c) Continuité du sens

✓ Correct

Bonne coupure :
Chunk 1 : « Le Transformer encode le texte en représentations contextuelles. »
Chunk 2 : « L'attention multi-têtes capture les dépendances entre tous les tokens. »

✗ Incorrect

Mauvaise coupure :
Chunk 1 : « L'attention multi-têtes capture les »
Chunk 2 : « dépendances entre tous les tokens. » → Phrase brisée, sens perdu

IV. Pourquoi le chunking est-il critique ?

Sa qualité conditionne directement la précision de toute la chaîne RAG

Mauvais chunking → Mauvais embedding → Mauvais retrieval → Mauvaise réponse

Erreurs fréquentes à éviter

Couper par taille brute

Une définition ou une idée peut être coupée en plein milieu → phrase brisée, sens perdu.

Chunks trop grands

Un chunk trop dense contient plusieurs idées → l'embedding les 'moyenne' et perd en précision.

Aucun overlap

Les tokens aux frontières entre deux chunks perdent leur contexte → information manquante.

Ignorer la structure

Ne pas respecter les paragraphes, titres ou sections mélange des sujets dans un même chunk.

Méthode 01

Fixed-Size Chunking

Méthode 01 — Fixed-Size Chunking

Définition, paramètres et cas d'usage

Le Fixed-Size Chunking découpe le texte en segments de taille strictement fixe (N caractères ou N tokens), sans tenir compte des frontières sémantiques ou syntaxiques du document.

Paramètres clés

chunk_size	= 500	<i>Nombre de tokens par segment</i>
-------------------	--------------	-------------------------------------

chunk_overlap	= 0	<i>Pas de chevauchement entre segments</i>
----------------------	------------	--

separator	= \n	<i>Frontière de découpe préférée</i>
------------------	-------------	--------------------------------------

Quand l'utiliser ?

Prototypes et pipelines rapides

Documents sans structure particulière

Quand la vitesse prime sur la qualité

Point de départ avant d'affiner

Méthode 01 — Fixed-Size Chunking

Exemple sur texte RAG

Texte source — avant tout découpage

Le RAG est une architecture qui augmente les LLM avec une base de connaissances externe. Lorsqu'une question est posée, un retriever encode la requête en vecteur. Il compare ce vecteur aux embeddings des chunks indexés. Les K chunks les plus proches sont sélectionnés et injectés dans le prompt du LLM. Le LLM génère une réponse fondée sur des sources réelles.

↓ Découpe tous les 25 mots — taille fixe, sans analyse du contenu

Chunk 1 (mots 1–25)

Le RAG est une architecture qui augmente les LLM avec une base de connaissances externe. Lorsqu'une question est posée, un retriever encode

Chunk 2 (mots 26–50) — fragment !

la requête en vecteur. Il compare ce vecteur aux embeddings des chunks indexés. Les K chunks les plus proches sont sélectionnés. Ils sont

Question : « Comment le retriever encode-t-il la requête ? »

X Chunk 2 commence par « la requête » sans le sujet « encode ». Phrase brisée → réponse incomplète.

Méthode 01 — Fixed-Size Chunking

Code Python, avantages et limites

Code Python

```
from langchain.text_splitter import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap = 0,
    separator = "\n"
)

chunks = splitter.create_documents(
    [p["texte"] for p in pages]
)

print(f"Nombre de chunks : {len(chunks)}")
print(chunks[0].page_content)
```

Avantages

- ▶ Simple et rapide à implémenter
- ▶ Scalable sur de grands corpus
- ▶ Bon point de départ pour prototyper

Limites

- ▶ Coupe sans respecter le sens
- ▶ Peut briser une phrase ou une définition
- ▶ Frontières de découpe arbitraires

Méthode 02

Fixed-Size with Overlap

Méthode 02 — Fixed-Size with Overlap

Préserver le contexte aux frontières par chevauchement

Le Fixed-Size with Overlap ajoute une zone de chevauchement entre deux chunks consécutifs. Les tokens de l'overlap apparaissent dans les deux chunks, préservant le contexte aux frontières.

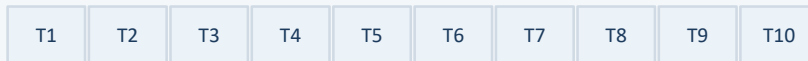
$$\text{overlap_ratio} = \frac{\text{overlap_size}}{\text{chunk_size}}$$

chunk_size = 500 *Taille de chaque chunk*

chunk_overlap = 100 *Tokens répétés — ratio ≈ 20 %*

Note : un overlap supérieur à 30 % entraîne une duplication excessive et surcharge inutilement la base vectorielle.

Visualisation — Sans overlap vs Avec overlap

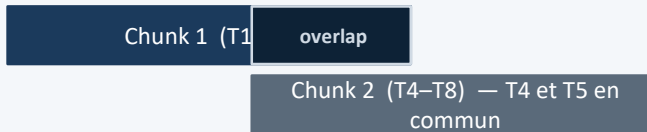


Sans overlap :



→ Rupture entre T5 et T6 — aucun contexte partagé

Avec overlap (2 tokens) :



→ T4 et T5 partagés — contexte préservé à la frontière

Méthode 02 — Fixed-Size with Overlap

Exemple sur texte RAG

Texte source — avant tout découpage

Le RAG est une architecture qui augmente les LLM avec une base de connaissances externe. Lorsqu'une question est posée, un retriever encode la requête en vecteur. Il compare ce vecteur aux embeddings des chunks indexés. Les K chunks les plus proches sont sélectionnés et injectés dans le prompt du LLM. Le LLM génère une réponse fondée sur des sources réelles.

↓ Découpe tous les 25 mots — overlap de 8 mots partagés entre Chunk 1 et Chunk 2

Chunk 1 (mots 1–25)

Le RAG est une architecture qui augmente les LLM avec une base de connaissances externe. Lorsqu'une question est posée, un retriever encode la requête en vecteur.

Chunk 2 (mots 18–42) — overlap inclus

un retriever encode la requête en vecteur. Il compare ce vecteur aux embeddings des chunks indexés. Les K chunks les plus proches sont sélectionnés.

Zone de chevauchement : « un retriever encode la requête en vecteur » est présent dans les deux chunks.

Question : « Comment le retriever encode-t-il la requête ? »

« encode la requête en vecteur » est présent dans les deux chunks → le contexte est complet → le LLM produit une réponse précise.

Méthode 02 — Fixed-Size with Overlap

Code Python, avantages et limites

Code Python

```
from langchain.text_splitter import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap = 100,      # overlap = 20 % de chunk_size
    separator = "\n"
)

chunks = splitter.create_documents(
    [p["texte"] for p in pages]
)

print(f"Nombre de chunks : {len(chunks)}")
# Note : plus de chunks qu'en méthode 1 à cause de l'overlap
```

Avantages

- Préserve le contexte aux frontières
- Simple à implémenter
- Très répandu en production

Limites

- Duplication de contenu
- Stockage vectoriel légèrement plus grand
- Redondance si overlap > 30 %

Méthode 03

Recursive Character Chunking

Méthode 03 — Recursive Character Chunking

Découpage en cascade selon la structure naturelle du texte

Le Recursive Character Chunking applique un découpage en cascade : il essaie d'abord de couper par paragraphe (`\n\n`), puis par ligne (`\n`), puis par phrase (`.`), puis par mot — jusqu'à obtenir des chunks de la taille cible.

Hiérarchie des séparateurs — du plus large au plus fin

1	<code>"\n\n"</code>	Paragraphe	→ Tentative 1 — séparateur prioritaire
2	<code>"\n"</code>	Ligne	→ Si le chunk reste trop long
3	<code>"."</code>	Phrase	→ Si le chunk reste encore trop long
4	<code>" "</code>	Mot	→ Si le chunk reste encore trop long
5	<code>" "</code>	Caractère	→ Dernier recours absolu

Méthode 03 — Recursive Character Chunking

Exemple : découpage en cascade sur un texte RAG

Texte source — paragraphe dense

Le RAG est une architecture qui augmente les LLM avec une base externe. Un retriever encode la requête en vecteur. Il compare aux embeddings via FAISS. Les K chunks proches sont sélectionnés. Ils alimentent le LLM. Le LLM réduit les hallucinations.

↓ Étape 1 : trop long → découpe par phrases (.)

Chunk A (phrases 1–2)

Le RAG est une architecture qui augmente les LLM avec une base externe. Un retriever encode la requête en vecteur dense.

Chunk B (phrases 3–4)

Il compare aux embeddings via FAISS. Les K chunks les plus proches sémantiquement sont sélectionnés.

↓ Phrases 5–6 encore trop longues → Étape 2 : découpe par mots

C1

Les chunks alimentent le LLM comme contexte.

C2

Le LLM génère une réponse ancrée dans les sources.

Question : « Comment le RAG réduit-il les hallucinations ? »

✓ C2 isolé sur cette info → embedding précis → retrieval pertinent → réponse correcte.

Méthode 03 — Recursive Character Chunking

Code Python, avantages et limites

Code Python

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap = 100,
    separators = [
        "\n\n", # 1. Paragraphe
        "\n", # 2. Ligne
        ".", # 3. Phrase
        " ", # 4. Mot
    ]
)

chunks = splitter.create_documents(
    [p["texte"] for p in pages]
)
```

Avantages

- ▶ Respecte la structure naturelle du texte
- ▶ Adaptatif : ajuste la découpe selon la taille
- ▶ Bon compromis qualité / vitesse

Limites

- ▶ Paramétrage des seuils peut être délicat
- ▶ Résultats variables selon le style de texte
- ▶ Chunks parfois de tailles très inégales

Méthode 04

Semantic Chunking

Méthode 04 — Semantic Chunking

Détecter les changements de sujet par similarité sémantique

Le Semantic Chunking place les frontières là où la similarité sémantique entre deux phrases consécutives chute fortement, signalant un changement de sujet. Il utilise un modèle d'embedding pour détecter ces transitions automatiquement.

Les 4 étapes de l'algorithme

1	Encoder chaque phrase	Un modèle d'embedding transforme chaque phrase en vecteur dense représentant son sens sémantique.
2	Calculer $\text{sim_cosine}(P_i, P_{i+1})$	On mesure la similarité cosinus entre chaque paire de phrases consécutives. Score ≈ 1 : même sujet. Score ≈ 0 : sujets différents.
3	Si $\text{sim} < \text{seuil } \theta \rightarrow \text{poser une frontière}$	Quand la similarité chute sous θ (ex : 0.70), une frontière de chunk est placée à cet endroit précis.
4	Regrouper les phrases en chunks	Les phrases entre deux frontières forment un chunk thématiquement cohérent — un seul sujet, embedding précis.

Méthode 04 — Semantic Chunking

Scores de similarité cosine et détection de frontière

Texte analysé : 5 phrases mêlant deux sujets — RAG (P1–P3) et Transformers (P4–P5)

P1	Le RAG est une architecture qui augmente les LLM avec une base de connaissances externe.	$\text{sim}(\text{P1}, \text{P2}) = 0.91$
P2	Un retriever encode la requête utilisateur en vecteur dense.	$\text{sim}(\text{P2}, \text{P3}) = 0.88$
P3	Il compare ce vecteur aux embeddings et sélectionne les K chunks.	$\text{sim}(\text{P3}, \text{P4}) = 0.42$
⌘ FRONTIÈRE DÉTECTÉE — $\text{sim} = 0.42 < \theta = 0.70$ — Changement de sujet : RAG → Transformers		
P4	Les transformers utilisent le mécanisme d'attention multi-têtes.	$\text{sim}(\text{P4}, \text{P5}) = 0.86$
P5	L'auto-attention capture les dépendances entre tous les tokens.	—

Chunk A = { P1, P2, P3 } — Sujet : RAG

Chunk B = { P4, P5 } — Sujet : Transformers

Méthode 04 — Semantic Chunking

Code Python

Code Python

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain.embeddings import HuggingFaceEmbeddings #Le découpeur a besoin d'un "cerveau" mathématique, pour
comprendre le sens des phrases.

# 1. Charger le modèle d'embedding
model = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2" #Son rôle est de prendre une phrase et de la transformer en un Vecteur
)

# 2. Créer le Semantic Chunker
splitter = SemanticChunker(
    model,
    breakpoint_threshold_type = "percentile",
    breakpoint_threshold_amount = 70
)

# 3. Appliquer sur les pages du PDF
chunks = splitter.create_documents(
    [p["texte"] for p in pages]
)
```

Méthode 04 — Semantic Chunking

Avantages et limites

Avantages

- ▶ Chunks thématiquement purs — un sujet par chunk
- ▶ Frontières naturelles et pertinentes
- ▶ Meilleur retrieval que Fixed-Size

Limites

- ▶ Nécessite un modèle d'embedding au chunking
- ▶ Plus lent sur de très grands corpus
- ▶ Seuil θ délicat à calibrer

Méthode 05

Agentic / Proposition Chunking

Méthode 05 — Agentic / Proposition Chunking

Décomposer le texte en propositions atomiques par LLM

L'Agentic Chunking utilise un LLM pour décomposer le texte en propositions atomiques — chaque chunk contient exactement une affirmation indépendante et vérifiable. C'est la méthode la plus précise et la plus coûteuse.

Comparaison avec les autres méthodes

Fixed-Size	Règle : <i>N tokens fixe</i>	✗ Fragment souvent incomplet
Recursive	Règle : <i>Séparateurs textuels</i>	✓ Chunk syntaxiquement correct
Semantic	Règle : <i>Similarité cosine</i>	✓ Chunk thématiquement cohérent
Agentic	Règle : <i>LLM analyse le sens</i>	✓ Proposition atomique et autonome

Méthode 05 — Agentic / Proposition Chunking

Exemple : décomposition en propositions atomiques

Texte source — un paragraphe dense

Le RAG est une architecture qui augmente les LLM avec une base de connaissances externe, sans réentraîner le modèle. Lorsqu'une question est posée, un retriever encode la requête en vecteur dense. Les K chunks les plus proches sont sélectionnés et injectés dans le prompt du LLM. Le LLM génère alors une réponse fondée sur ces sources réelles, réduisant ainsi les hallucinations.

↓ Le LLM décompose chaque affirmation en proposition atomique indépendante

P1 — Le RAG est une architecture d'IA.

P2 — Le RAG augmente les LLM avec une base de connaissances externe.

P3 — Le RAG ne nécessite pas de réentraîner le modèle.

P4 — Le retriever encode la requête utilisateur en vecteur dense.

P5 — Les K chunks les plus proches sont sélectionnés par similarité.

P6 — Le RAG réduit les hallucinations du LLM.

Question : « Est-ce que le RAG réduit les hallucinations ? »

✓ P6 retourné directement — proposition atomique, réponse exacte, sans bruit contextuel.

Méthode 05 — Agentic / Proposition Chunking

Code Python, avantages et limites

Code Python

```
# Principe : appel LLM pour chaque passage

prompt = """Décompose ce texte en propositions atomiques.
Chaque proposition doit être indépendante et complète."""

# Pour chaque passage du document
propositions = llm.invoke(prompt + texte)

# Chaque proposition devient un chunk distinct
# → embedding précis, retrieval ciblé
```

Avantages

- ▶ Chunks atomiques et totalement autonomes
- ▶ Meilleure précision du retrieval
- ▶ Idéal pour les systèmes Q&A factuels

Limites

- ▶ Très coûteux — un appel LLM par chunk
- ▶ Lent sur de grands documents
- ▶ Qualité dépend du LLM utilisé

Tableau Comparatif — 5 Méthodes

Choisir la bonne méthode selon le contexte d'usage

Méthode	Complexité	Qualité	Overlap	Coût	Cas d'usage
Fixed-Size	Faible	Moyenne	Non	Faible	Prototypes, pipelines rapides
Fixed-Size + Overlap	Faible	Bonne	Oui	Faible	Point de départ recommandé
Recursive	Modérée	Très bonne	Oui	Moyen	Textes variés, docs académiques
Semantic	Élevée	Excell.	Non	Moyen	Production, docs hétérogènes
Agentic	Élevée	Max.	Non	Élevé	Q&A factuel, haute précision

Conseil : Commencer par Fixed-Size + Overlap, puis affiner vers Recursive → Semantic selon la qualité des résultats.

Synthèse — Points-Clés à Retenir

Ce qu'il faut maîtriser à l'issue de ce cours

1	Le chunking est la brique fondamentale du RAG	Sa qualité détermine la précision des embeddings, la pertinence du retrieval et la qualité des réponses.
2	Un bon chunk = une seule idée cohérente	Cohérence sémantique, taille adaptée et continuité du sens sont les trois critères essentiels.
3	L'overlap préserve le contexte aux frontières	Un ratio de 10–20 % est recommandé pour éviter les pertes sans dupliquer excessivement.
4	5 méthodes — choisir selon le contexte	Fixed-Size pour les prototypes, Recursive pour les docs structurés, Semantic et Agentic pour la production.
5	Progresser par itération	Commencer par Fixed-Size + Overlap, puis affiner selon les résultats de retrieval observés.

Conclusion

Chunking dans les Systèmes RAG

Programme 4IASDR

1

Le chunking est la première brique critique de tout système RAG

2

Un bon chunk = une idée cohérente à la bonne taille

3

L'overlap protège le contexte aux frontières — à toujours activer

4

5 méthodes — le choix dépend du document et du contexte

5

Simuler des queries est la meilleure façon de valider un chunking

6

Commencer simple, affiner progressivement